

Today 11:41 AM

Lab\_4\_VibeCodin...  
PDF

help me fork and clone it, and guide me on how to do this lab project

Absolutely. We can do this **step-by-step from the very beginning**, including forking/cloning, running the game, fixing the broken parts with Vibe Coding, adding every feature in the README, making separate commits, and finally preparing the Lab-4 submission.

The handout says the workflow is: **go to your assigned repo → read README → fork/clone → run the Python game/puzzle → record before → use prompts to fix/add features → record after → push to your repo, with separate commits for each task.**  Lab\_4\_VibeCoding\_Student\_handout

**First, I need one thing from you**

**Send me the GitHub repository link that was assigned to you.**

It will probably look something like:

</> Plain text

`https://github.com/SETAPESU26/.....`

Once you send it, we'll do this **one step at a time**:

1. **Fork the repo** to your personal GitHub.
2. **Clone your fork** onto your Mac.
3. Open it in **VS Code**.
4. Read and understand the `README.md`.
5. Run the original Python game/puzzle.
6. Make the required **10-second before video**.
7. Identify the broken code/features.
8. I'll give you the **exact Vibe Coding prompts** to use — aiming to solve each task within the required 3–4 attempts.
9. Implement each README feature **one by one**.
10. Make a **separate Git commit for every task**.
11. Push everything to **your fork**, not the original SETAPESU26 repo.
12. Make the **10-second after video**.
13. Put everything under your **Lab-4** folder as required: videos, updated code, and exported ChatGPT/Vibe Coding history.  Lab\_4\_VibeCoding\_Student\_handout

**Don't start changing anything yet.** Send me the assigned GitHub repo link first, and I'll guide you through **Step 1: Forking it**.

finished forking then

Perfect. Since you've **already forked it**, let's move to cloning your fork onto your Mac.

## Step 2 — Copy the URL of YOUR fork

Go to your fork on GitHub.

You should now be on something like:

</> Plain text

```
https://github.com/YOUR-USERNAME/REPO-NAME
```

Click the green **Code** button → select **HTTPS** → copy the URL.

It should look like:

</> Plain text

```
https://github.com/yourusername/repository-name.git
```

## Step 3 — Clone it

Open **Terminal** on your Mac.

First, go to the folder where you want your lab project. For example, if you keep college work in Documents:

</> Bash

```
cd ~/Documents
```

Then clone:

</> Bash

```
git clone https://github.com/YOUR-USERNAME/REPO-NAME.git
```

Replace the URL with the one you copied.

You should see something like:

`</>` Plain text

```
Cloning into 'REPO-NAME'...
remote: Enumerating objects...
Receiving objects: 100% ...
Resolving deltas: 100% ...
```

## Step 4 — Enter the project

`</>` Bash

```
cd REPO-NAME
```

Then check:

`</>` Bash

```
git status
```

You should see something like:

`</>` Plain text

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

## Step 5 — Open it in VS Code

From inside the repository folder:

`</>` Bash

```
code .
```

If `code .` doesn't work, tell me what error you get and we'll fix it.

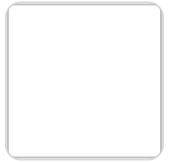
---

**Stop here after you've opened the repo in VS Code.**

Then tell me **“opened”**, and we'll inspect the `README.md` together **before running or changing anything**. This is important because the README contains the actual features you're required to

implement. The lab specifically says to go through the README before proceeding.

 Lab\_4\_VibeCoding\_Student\_handout



this is the directory

Yes, perfect. 👍 Your desired folder is:

 Plain text

Macintosh HD

```
└─ Users
   └─ viswaja
      └─ Documents
         └─ sem 5
            └─ LABS
               └─ SE Lab
                  └─ SE Lab-4
```

So let's clone the repo **directly inside SE Lab-4** .

## Step 1 — Open Terminal

Run:

 Bash

```
cd ~/Documents/"sem 5"/LABS/"SE Lab"/"SE Lab-4"
```

Then verify you're in the correct location:

 Bash

```
pwd
```

You should get approximately:

 Plain text

```
/Users/viswaja/Documents/sem 5/LABS/SE Lab/SE Lab-4
```

## Step 2 — Clone your fork here

Now use the URL of **your fork**, not the original SETAPESU26 repo:

</> Bash

```
git clone YOUR-FORK-URL
```

For example:

</> Bash

```
git clone https://github.com/yourusername/game-repo.git
```

### Step 3 — Check Finder

After cloning, your folder should look like:

</> Plain text

```
SE Lab-4
├── game-repo
│   ├── README.md
│   ├── ...
│   └── ...
```

**Don't create another Lab-4 folder yet.** The cloned repository itself is your working project.

### Step 4 — Open the cloned repo in VS Code

Once cloning finishes:

</> Bash

```
cd game-repo
code .
```

Replace `game-repo` with the actual repository folder name.

---

**Do this first and send me the output of `pwd` and `git status`** (or a screenshot of the VS Code folder). Then we'll inspect the README and figure out exactly what your lab requires **before** changing any code.

```
(base) viswaja@Viswajas-MacBook-Air SE Lab-4 % git clone
🔗 https://github.com/sriviswaja/49_brick-breaker.git
```

```
Cloning into '49_brick-breaker'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 15 (delta 2), reused 15 (delta 2), pack-reused 0
(from 0)
Receiving objects: 100% (15/15), 9.57 KiB | 9.57 MiB/s, done.
Resolving deltas: 100% (2/2), done.
(base) viswaja@Viswajas-MacBook-Air SE Lab-4 % cd game-
repo
code .
cd: no such file or directory: game-repo
zsh: command not found: code
```

You're actually **good** — the cloning worked perfectly. 🎉

The only problems are:

1. You cloned it as `49_brick-breaker` , not `game-repo` .
2. `code` . isn't installed/enabled in your Terminal yet.

## 1. Enter your actual cloned folder

You're currently here:

🔗 Plain text

SE Lab-4

Run:

🔗 Bash

cd 49\_brick-breaker

Then:

🔗 Bash

pwd

You should see:

</> Plain text

```
/Users/viswaja/Documents/sem 5/LABS/SE Lab/SE Lab-4/49_brick-breaker
```

Now check Git:

</> Bash

```
git status
```

You should get something like:

</> Plain text

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

## 2. Don't worry about `code .` yet

This:

</> Plain text

```
zsh: command not found: code
```

just means the VS Code command-line shortcut isn't configured. **It does NOT mean VS Code isn't installed.**

For now, easiest option:

- Open **VS Code normally**
- Go to **File → Open Folder...**
- Navigate to:

</> Plain text

```
Documents
→ sem 5
→ LABS
→ SE Lab
→ SE Lab-4
→ 49_brick-breaker
```

- Click **Open**

### 3. Then show me the README

Once the repo is open in VS Code, you'll see something like:

</> Plain text

49\_brick-breaker

├─ README.md

└─ ...

Open **README.md** and send me a screenshot of it (or paste its contents here).

Don't modify anything yet.

We'll first understand **exactly what the original game does, what's broken, and every feature the README asks us to add**. Then I'll guide you through the lab task-by-task, including the required separate commits. The handout specifically requires separate commits for each task and says not to create a PR to the main `SETAPESU26` repository.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)



**README.md**  
Document

Perfect — this is exactly what we needed. I've read the README. Your project is a **Pygame Brick Breaker**, and there are **4 required tasks**.  [README](#)

Before we touch the code, let's make sure we follow the lab exactly.

### Your lab roadmap

#### ● Task 0 — Run the original version

The README says the project requires Python 3.10+ and:

</> Bash

```
pip install -r requirements.txt
python main.py
```

 [README](#)

**Do NOT modify anything yet.**

We need to see the broken behavior first because the submission requires a **10-second video before changes showing the bug/broken behavior**.  [README](#)

## 🟡 Task 1 — Collision Detection

Current problem:

The ball always bounces vertically from the paddle/bricks, even when it hits the brick from the left or right.

We need to improve collision detection so the ball's direction reflects **which side of the brick was actually hit**. [📖 README](#)

We'll use an LLM prompt for this, because the lab specifically wants us to use AI as a debugging/pair-programming partner. [📖 README](#)

---

## 🟡 Task 2 — Game Over

When the game ends:

- Display **WIN** if all bricks are destroyed.
  - Display **LOSS** if all lives are gone.
  - Show the **final score**.
  - Wait for user input rather than simply printing something in Terminal. [📖 README](#)
- 

## 🟢 Task 3 — Replay

After the game ends:

```
</> Plain text
```

```
GAME OVER
```

```
Final Score: 240
```

```
Play Again?
```

```
[1] Easy
```

```
[2] Medium
```

```
[3] Hard
```

```
[4] Exit
```

The exact UI doesn't have to look like that; the important requirement is that the player can select **Easy / Medium / Hard** or exit. The difficulty should affect ball speed/paddle size. [📖 README](#)

---

## 🟢 Task 4 — Sound

Add sound effects for:

- 🎮 Brick breaking
- 🎮 Ball hitting paddle/wall
- 🎮 Win/game-over moment

 [README](#)

---

## But first: Task 0

Let's **not** use AI to change anything yet.

Open Terminal and go into the repo:

```
</> Bash
```

```
cd ~/Documents/"sem 5"/LABS/"SE Lab"/"SE Lab-4"/49_brick-breaker
```

Then:

```
</> Bash
```

```
ls
```

You should see roughly:

```
</> Plain text
```

```
README.md  
main.py  
requirements.txt  
game
```

Then install the dependencies:

```
</> Bash
```

```
pip install -r requirements.txt
```

If that succeeds, run:

```
</> Bash
```

```
python main.py
```

## ⚠ Important

If `python main.py` doesn't work, **don't start randomly changing things**.

Copy the entire error here and I'll tell you exactly what to do.

---

## Then we need your BEFORE video 🎥

Once the game runs, play it for a little while and specifically try to reproduce the collision bug described in Task 1.

You're looking for something like:

**Ball → hits the side of brick → ball incorrectly bounces vertically**

The README says your before video should show the **bug/broken behavior**, not just the game sitting on the screen.  README

Record about **10 seconds**.

Don't worry about editing the video yet.

---

## One important thing about the Git workflow

We're going to make **separate commits**:

 Plain text

```
Task 1 - Fix collision detection
Task 2 - Add game over screen
Task 3 - Add replay and difficulty
Task 4 - Add sound effects
```

That matches the lab requirement for separate commits for each task.

 Lab\_4\_VibeCoding\_Student\_handout

And because you cloned **your fork**, all our pushes will go to:

 Plain text

```
sriviswaja/49_brick-breaker
```

—not the original repository.

---

## Do this now

Run these **three commands**:

```
</> Bash
```

```
cd ~/Documents/"sem 5"/LABS/"SE Lab"/"SE Lab-4"/49_brick-breaker
```

```
</> Bash
```

```
pip install -r requirements.txt
```

```
</> Bash
```

```
python main.py
```

**Tell me exactly what happens when you run `python main.py`** (or paste the error if you get one).

Then we'll handle the **before video** → **Task 1**, and I'll give you the exact first Vibe Coding prompt to use.

```
(base) viswaja@Viswajas-MacBook-Air 49_brick-breaker % pip
install -r requirements.txt
python main.py
Collecting pygame (from -r requirements.txt (line 1))
  Downloading pygame-2.6.1-cp313-cp313-
macosx_11_0_arm64.whl.metadata (12 kB)
Downloading pygame-2.6.1-cp313-cp313-
macosx_11_0_arm64.whl (12.4 MB)
```

---

```
12.4/12.4 MB 17.7 MB/s 0:00:00
```

```
Installing collected packages: pygame
Successfully installed pygame-2.6.1
pygame 2.6.1 (SDL 2.28.4, Python 3.13.13)
Hello from the pygame community.
Ⓢ https://www.pygame.org/contribute.html
Game over! Final score: 9
```

```
played the game
```

Perfect.  The game is running successfully, and you've already played the original version.

You got:

</> Plain text

```
pygame 2.6.1
Python 3.13.13
Game over! Final score: 9
```

So **setup is complete**. Don't change any code yet.

## Step 0.5 — Record the BEFORE video 🎥

The README specifically asks for a **10-second video before your changes showing the bug/broken behavior**. [📖 README](#)

Since you already played once, let's reproduce the collision issue if possible.

### Try this

Run:

</> Bash

```
python main.py
```

Play for around 10 seconds and watch specifically when the ball hits a brick.

The bug we're supposed to address is:

When the ball hits the **left or right side** of a brick, it doesn't properly change its horizontal direction; the bounce can remain vertical/incorrect. [📖 README](#)

If you can capture that happening, **record that ~10-second section**.

If the game ends too quickly or you can't clearly reproduce the side collision, that's okay—don't spend ages trying. Record a short gameplay clip showing the original behavior.

### Important

**Do not modify the code before this video.**

Name it something simple, for example:

</> Plain text

```
before.mp4
```

Keep it somewhere accessible for your submission later.

---

## Then we're starting Task 1 🚀

Once you've recorded the video, **don't immediately edit the code yourself.**

The lab specifically wants you to use an LLM as a debugging/pair-programming partner and complete the tasks iteratively.  [README](#)

We're going to make our first AI prompt deliberately good so we have a chance of solving it in **1–3 attempts**, rather than repeatedly asking vague questions.

### First, open the project in VS Code

Your folder is:

 Plain text

```
SE Lab-4
├── 49_brick-breaker
│   ├── main.py
│   ├── requirements.txt
│   ├── README.md
│   └── game
│       ├── game_engine.py
│       ├── paddle.py
│       ├── ball.py
│       └── brick.py
```

The README confirms this structure.  [README](#)

Open the relevant Python files in VS Code.

### For Task 1, we're going to inspect:

 Plain text

```
game/ball.py
game/brick.py
game/game_engine.py
```

### Don't change anything yet.

After you've recorded `before.mp4` , send me either:

- a screenshot of the files/code in VS Code, or

- paste the contents of `ball.py` and `game_engine.py` .

Then I'll help you formulate the **first Vibe Coding prompt for Task 1**, and we'll keep track of exactly what prompt you used because your final submission requires the complete chat/LLM history.

 README

**So right now: record the ~10-second BEFORE video, then tell me before video done .**

```
import pygame

class Ball:
    def __init__(self, x, y, radius=8):
        self.x = x
        self.y = y
        self.radius = radius
        self.vx = 0
        self.vy = 0

    def move(self):
        self.x += self.vx
        self.y += self.vy

    def rect(self):
        return pygame.Rect(self.x - self.radius, self.y - self.radius,
self.radius * 2, self.radius * 2)

import pygame
from .paddle import Paddle
from .ball import Ball
from .brick import Brick

# Game Engine

WHITE = (255, 255, 255)
BG = (15, 15, 25)
BRICK_COLORS = [
    (200, 60, 60),
    (200, 140, 60),
    (200, 200, 60),
```

```
(80, 180, 80),  
(80, 140, 200),  
]
```

```
class GameEngine:
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)
```

```
        self.ball = Ball(width // 2, height - 50, radius=8)
```

```
        self.ball.vx, self.ball.vy = 4, -4
```

```
        self.rows, self.cols = 5, 8
```

```
        self.bricks = self._build_bricks(self.rows, self.cols)
```

```
        self.lives = 3
```

```
        self.score = 0
```

```
        self.font = pygame.font.SysFont("Arial", 28)
```

```
        self.game_over = False
```

```
        self.result = None # "win" or "lose"
```

```
    def _build_bricks(self, rows, cols):
```

```
        bricks = []
```

```
        margin, gap, top = 30, 6, 60
```

```
        brick_w = (self.width - margin * 2 - gap * (cols - 1)) // cols
```

```
        brick_h = 22
```

```
        for r in range(rows):
```

```
            for c in range(cols):
```

```
                x = margin + c * (brick_w + gap)
```

```
                y = top + r * (brick_h + gap)
```

```
                bricks.append(Brick(x, y, brick_w, brick_h))
```

```
        return bricks
```

```
    def handle_event(self, event):
```

```
        # This game only needs continuously-held-key input for the
```

```
        # paddle, handled in handle_input each frame.
```

```
        pass
```

```
def handle_input(self):
    if self.game_over:
        return
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.paddle.move(-self.paddle.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.paddle.move(self.paddle.speed, self.width)

def update(self):
    if self.game_over:
        return

    self.ball.move()

    if self.ball.x - self.ball.radius <= 0 or self.ball.x +
self.ball.radius >= self.width:
        self.ball.vx *= -1
    if self.ball.y - self.ball.radius <= 0:
        self.ball.vy *= -1

    if self.ball.rect().colliderect(self.paddle.rect()):
        # NOTE: always flips the ball's vertical velocity on a
        # paddle collision, regardless of which side of the paddle
        # was actually hit. See Task 1 in the README.
        self.ball.vy *= -1

    for brick in self.bricks:
        if brick.alive and self.ball.rect().colliderect(brick.rect()):
            brick.alive = False
            self.score += 1
            # NOTE: same unconditional vertical-velocity flip as
            # the paddle collision above - a brick hit from the
            # left or right should redirect the ball sideways
            # (flip vx), but this always flips vy instead. See
            # Task 1 in the README.
            self.ball.vy *= -1
            break

    if self.ball.y - self.ball.radius > self.height:
```

```
self.lives -= 1
if self.lives <= 0:
    self.game_over = True
    self.result = "lose"
else:
    self._reset_ball()

if all(not b.alive for b in self.bricks):
    self.game_over = True
    self.result = "win"

def _reset_ball(self):
    self.ball.x, self.ball.y = self.width // 2, self.height - 50
    self.ball.vx, self.ball.vy = 4, -4

def render(self, screen):
    screen.fill(BG)

    pygame.draw.rect(screen, WHITE, self.paddle.rect())
    pygame.draw.circle(screen, WHITE, (int(self.ball.x),
int(self.ball.y)), self.ball.radius)

    for i, brick in enumerate(self.bricks):
        if brick.alive:
            row = i // self.cols
            color = BRICK_COLORS[row % len(BRICK_COLORS)]
            pygame.draw.rect(screen, color, brick.rect())

    score_text = self.font.render(f"Score: {self.score}", True,
WHITE)
    screen.blit(score_text, (10, 10))
    lives_text = self.font.render(f"Lives: {self.lives}", True,
WHITE)
    screen.blit(lives_text, (self.width - 130, 10))

    if self.game_over and not getattr(self, "_game_over_logged",
False):
        # NOTE: no proper end screen yet - see Task 2 in the
README.
        if self.result == "win":
```

```

        print("You win! Final score:", self.score)
    else:
        print("Game over! Final score:", self.score)
    self._game_over_logged = True

```

Yes — this is enough to understand **exactly what Task 1 is asking us to fix**. And importantly, your code already has comments that explicitly mark the intended bug.

## What is currently wrong?

In `update()` :

`</>` Python

```

if self.ball.rect().colliderect(self.paddle.rect()):
    self.ball.vy *= -1

```

and for bricks:

`</>` Python

```

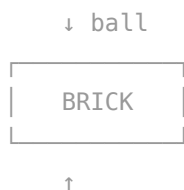
if brick.alive and self.ball.rect().colliderect(brick.rect()):
    brick.alive = False
    self.score += 1
    self.ball.vy *= -1

```

The problem is that `colliderect()` only tells us **"these two rectangles overlap."** It doesn't tell the game which side was hit.

So currently:

`</>` Plain text



flips `vy` .

But if the ball comes from the side:

`</>` Plain text

→ ball



we need to flip `vx` instead.

That's exactly the Task 1 requirement in the README. [📖 README](#)

---

## Now comes the important Vibe Coding part

Don't manually fix it yet.

The lab specifically asks you to use an LLM as your debugging/pair-programming partner and use an iterative process for each task. [📖 README](#)

Since **this conversation itself is your LLM interaction**, we can use the prompt below as your first Task 1 prompt.

### 🟡 Task 1 — Attempt 1

Send me/use this prompt:

**I am working on a Pygame Brick Breaker game. I need to fix the collision detection for Task 1 without changing unrelated functionality.**

In `game/game_engine.py`, the ball currently always reverses `vy` when it collides with the paddle or a brick. This is incorrect when the ball hits the left or right side.

Please analyze the existing collision code and modify it so that:

- A collision with the top or bottom side of a brick/paddle reverses `vy`.
- A collision with the left or right side reverses `vx`.
- The brick should still be destroyed and the score should still increase.
- Existing paddle movement, wall collisions, lives, and win/loss behavior should remain unchanged.
- Avoid unnecessary changes to other files.

Explain briefly how you determine which side of the rectangle was hit, then provide the exact code changes.

### Why this is a good prompt

We're giving the AI:

- **the problem**
- **where the problem is**
- **what the correct behavior should be**

- **what must NOT break**
- **a constraint against unnecessary changes**

That's much better than:

"Fix the collision detection."

And it gives us a clear record for the lab's **LLM/chat-history requirement**. The README asks for the complete chat history in the submission.  [README](#)

---

## BUT — before applying the answer

I want you to **paste that prompt here first** rather than blindly applying whatever an AI coding tool gives you.

I'll act as the reviewer.

We'll do:

 Plain text

Attempt 1

↓

AI suggestion

↓

We inspect it

↓

Apply only if correct

↓

Run game

↓

Test collision

↓

If needed → Attempt 2

This also satisfies the spirit of the lab: **AI generates/suggests, you critically review and test it.**

## One thing I noticed

Your `GameEngine` also has:

 Python

```
self.game_over = False
self.result = None
```

and the `render()` method currently only prints:

 Python

```
print("You win! Final score:", self.score)
```

or:

 Python

```
print("Game over! Final score:", self.score)
```

That is **Task 2**, so **leave it alone for now**. We'll handle it in its own commit after Task 1. The README explicitly separates Task 1 collision detection from Task 2 game-over behavior.  [README](#)

---

## So for now:

Use the Task 1 prompt above.

If you're using ChatGPT directly, you've effectively already given it to me — but for your lab record, you should keep this conversation as your LLM history.

Next, I'll help you evaluate the proposed code **before you paste it into the project**.

**I am working on a Pygame Brick Breaker game. I need to fix the collision detection for Task 1 without changing unrelated functionality.**

In `game/game_engine.py`, the ball currently always reverses `vy` when it collides with the paddle or a brick. This is incorrect when the ball hits the left or right side.

Please analyze the existing collision code and modify it so that:

- A collision with the top or bottom side of a brick/paddle reverses `vy`.
- A collision with the left or right side reverses `vx`.
- The brick should still be destroyed and the score should still increase.
- Existing paddle movement, wall collisions, lives, and win/loss behavior should remain unchanged.
- Avoid unnecessary changes to other files.

Explain briefly how you determine which side of the rectangle was hit, then provide the exact code changes.

Yes. For **Attempt 1**, this is the kind of solution we should expect the AI to propose.

The key idea is to remember the ball's position **before it moved**, then compare that previous rectangle with the rectangle it collided with. That lets us determine whether the ball approached from the left/right or from above/below.

### Suggested change in `game_engine.py`

Replace your current `update()` method with this:

`</>` Python

```
def update(self):
    if self.game_over:
        return

    # Store the ball's position before moving
    previous_rect = self.ball.rect()

    self.ball.move()

    if self.ball.x - self.ball.radius <= 0 or self.ball.x + self.ball.radius >= self.width:
        self.ball.vx *= -1
    if self.ball.y - self.ball.radius <= 0:
        self.ball.vy *= -1

    # Paddle collision
    if self.ball.rect().collidect(self.paddle.rect()):
        current_rect = self.ball.rect()
        paddle_rect = self.paddle.rect()

        # Determine whether the ball hit the horizontal or vertical side
        if current_rect.x < paddle_rect.x && current_rect.x + current_rect.width > paddle_rect.x + paddle_rect.width:
            self.ball.vx *= -1
        else:
            self.ball.vy *= -1
```

### What changed?

Previously:

`</>` Python

```
self.ball.vy *= -1
```

was used **for every collision**.

Now we ask:

Plain text

Where was the ball before it moved?

↓

Did it approach from the left/right?

YES → reverse vx

NO → reverse vy

For example:

Plain text

Ball moving RIGHT

● →



Previous `ball.right <= brick.left`

↓

`vx *= -1`

So the ball goes back to the left.

For a vertical collision:

Plain text

●

↓



Previous position indicates  
the ball approached from above

↓

`vy *= -1`

## One small cleanup

The suggested code has:

Python

```
current_rect = self.ball.rect()
```

but doesn't actually use `current_rect` .

So I'd remove those two lines:

</> Python

```
current_rect = self.ball.rect()
```

They're unnecessary.

---

## Before we accept Attempt 1

There is one thing I want you to test rather than blindly accepting it.

The requirement says:

"A collision with the top or bottom side of a brick/paddle reverses `vy` ; a collision with the left or right side reverses `vx` ."

The proposed solution uses the **previous position** to infer the approach direction. That's reasonable for this game because the ball moves only 4 pixels per frame.

So let's treat this as **Attempt 1**.

## Do this now

In VS Code:

1. Open `game/game_engine.py` .
2. Replace **only the `update()` method** with the version above.
3. Remove the unused `current_rect` lines.
4. Save.

Then run:

</> Bash

```
python main.py
```

## Test specifically:

Try to make the ball hit:

1. **The top of a brick** → should bounce vertically.

2. **The side of a brick** → should reverse horizontally.
3. **The paddle** → should still bounce correctly.
4. **The walls** → should still work.
5. Destroy a brick → score should still increase.

**Do not commit yet.**

If it works, we'll make your first commit:

 Bash

```
git add game/game_engine.py
git commit -m "Fix brick and paddle collision detection"
git push origin main
```

That becomes your **Task 1 commit**.

If something behaves incorrectly, tell me exactly what happened and we'll make **Attempt 2** rather than pretending Attempt 1 worked. This is also useful for your required complete LLM chat history.

 README

Traceback (most recent call last):

```
File "/Users/viswaja/Documents/sem 5/LABS/SE Lab/SE Lab-4/49_brick-breaker/game/game_engine.py", line 2, in <module>
    from .paddle import Paddle
ImportError: attempted relative import with no known parent package
```

Yep — this error is because you **ran game\_engine.py directly**.

The file contains:

 Python

```
from .paddle import Paddle
from .ball import Ball
from .brick import Brick
```

The `.` means "import from the `game` package." Python only understands that when the file is being run as part of the package.

**Don't change the imports.**

Your project is designed to be started through `main.py`, as the README says:

```
</> Bash  
python main.py
```

 README

## Do this instead

In Terminal, go back to the **root project folder**:

```
</> Bash  
cd ~/Documents/"sem 5"/LABS/"SE Lab"/"SE Lab-4"/49_brick-breaker
```

Check:

```
</> Bash  
pwd
```

It should end with:

```
</> Plain text  
/49_brick-breaker
```

Then run:

```
</> Bash  
python main.py
```

## Why this happened

You probably did something like:

```
</> Bash  
python game/game_engine.py
```

 Don't do that.

Use:

```
</> Bash
```

python main.py

✅ main.py imports GameEngine correctly, and Python then knows game is a package.

## One more thing

If the game now starts, **test the collision behavior**.

If you get another error, paste the entire traceback here.

If the game works, **don't commit yet** — tell me what happened when you tested the brick collisions, and we'll decide whether Task 1 Attempt 1 actually passed.

okay got before after video, now what. do i save it in the SE lab 4 folder?

Yes — but **let's organize it properly now** so your final submission is clean.

The handout says the final deliverables need to go under a **Lab-4 folder** and include the before/after videos, updated code, and chat history.  Lab\_4\_VibeCoding\_Student\_handout The README also specifically requires the before and after 10-second videos and the complete LLM chat history.

 README

## Your current structure

Right now you have:

</> Plain text

```
SE Lab-4/
├── 49_brick-breaker/ ← GitHub repository
│   ├── main.py
│   ├── README.md
│   ├── requirements.txt
│   └── game/
```

**Don't put before.mp4 and after.mp4 directly inside the GitHub repo yet.**

Instead, create a submission folder alongside the repo:

</> Plain text

```
SE Lab-4/
|
├── 49_brick-breaker/ ← your cloned GitHub repo
```

```
|  
└─ Lab-4/ ← submission folder  
    └─ before.mp4  
    └─ after.mp4  
    └─ chat-history.pdf
```

This keeps your actual project repository clean while giving you the required `Lab-4` submission folder.

## So do this now

Inside:

</> Plain text

```
SE Lab-4
```

create a folder called:

</> Plain text

```
Lab-4
```

Then put your two videos there:

</> Plain text

```
Lab-4/  
└─ before.mp4  
└─ after.mp4
```

**Don't worry about `chat-history.pdf` yet.** We'll handle that at the very end after all 4 tasks are completed.

---

## But there's one important question

You said **"got before after video"**.

If by **after video** you mean you recorded it **after only fixing Task 1**, that's actually okay — but it is **not your final after video**.

The README says the after video should show the **bug fixed and the new features working**.

 README

We still have:

- ☒ Task 1 — Collision detection
- ☐ Task 2 — Game Over screen
- ☐ Task 3 — Replay + difficulty
- ☐ Task 4 — Sound

So **don't treat the current after video as the final one**. We will record the final `after.mp4` again after Tasks 1–4 are finished.

**Right now our next step is:**

**Task 1 → verify it works → commit it.**

Run:

```
</> Bash
```

```
cd ~/Documents/"sem 5"/LABS/"SE Lab"/"SE Lab-4"/49_brick-breaker
```

Then:

```
</> Bash
```

```
git status
```

Send me the output.

We'll make sure **only the Task 1 change** is present, then I'll walk you through your **first Git commit and push**.

i created lab 4 in a SE-Lab-288 (which is my github repo for se labs) to keep all programs, i can move it there

Yes, **that's actually a better organization** if `SE-Lab-288` is your main GitHub repo where you keep all your SE lab work.

You can move the **submission materials** there, but I recommend keeping the cloned `49_brick-breaker` repository separate because that folder is itself a Git repository.

Your structure can be:

```
</> Plain text
```

```
SE-Lab-288/  
└─ Lab-4/
```

```

├─ before.mp4
├─ after.mp4
└─ 49_brick-breaker/
    ├─ README.md
    ├─ main.py
    ├─ requirements.txt
    └─ game/

```

### ⚠ One important Git point

Don't put the entire `49_brick-breaker` Git repository **inside another Git repository** unless you intentionally want a submodule. Since you're doing this as a student lab, that's unnecessary and can get confusing.

So I'd recommend:

</> Plain text

```

Documents/
├─ sem 5/
│   └─ LABS/
│       └─ SE Lab/
│           ├── SE-Lab-288/ ← your main SE lab repo
│           │   └─ Lab-4/
│           │       ├── before.mp4
│           │       └─ after.mp4
│           └─ SE Lab-4/
│               └─ 49_brick-breaker/ ← separate GitHub repo/fork

```

This is cleaner.

### For your final submission

Your `SE-Lab-288` repo can eventually have:

</> Plain text

```

SE-Lab-288/
├─ Lab-1/
├─ Lab-2/
├─ Lab-3/
├─ Lab-4/
│   ├── before.mp4
│   ├── after.mp4
│   └─ chat-history.pdf
└─ ...

```

And your `49_brick-breaker` fork remains its own repo containing the **actual updated code**.

The lab handout says the Lab-4 submission needs the videos, updated code, and chat history under the Lab-4 folder.  `Lab_4_VibeCoding_Student_handout`

**One correction to what I said earlier:** don't worry about physically putting the cloned Git repo inside `SE-Lab-288` ; keeping the two repositories separate is safer.

For now, put your **before video** somewhere safe, and we'll eventually put the **final after video** and chat-history into `SE-Lab-288/Lab-4` .

Next: let's verify **Task 1**, then make the first commit to your `49_brick-breaker` fork.



okay now i need to upload chat history as a pdf in this folder and ill be done right? how do i get chat history

Yes — you're **very close**. Your `Lab 4` folder currently has:

</> Plain text

```
Lab 4/  
├─ 49_brick-breaker/  
├─ brick_breaker_after  
└─ brick_breaker_before
```

The handout requires **videos + updated code + complete chat history**.

 `Lab_4_VibeCoding_Student_handout`

### One important Git issue first ⚠️

Because `49_brick-breaker` is itself a Git repository, if you put it inside your `SE-Lab-AM288` repository, Git may treat it as a **nested repository** rather than tracking all its files.

So before we call it completely done, we'll make sure the updated code is actually included in `SE-Lab-AM288` .

---

## Getting the Chat History PDF

Since this entire conversation is your LLM interaction, the easiest method is to **print this ChatGPT conversation to PDF**.

## On your Mac in Chrome:

1. Stay on this ChatGPT conversation.
2. Scroll through the conversation once so the messages are loaded.
3. Press:

⌘ Plain text

⌘ Command + P

4. The Chrome print window should appear.
5. For **Destination**, choose:

⌘ Plain text

Save to PDF

6. Choose a filename such as:

⌘ Plain text

Lab4\_Chat\_History.pdf

7. Save it.

Then put it here:

⌘ Plain text

SE-Lab-AM288/

└─ Lab 4/

├─ brick\_breaker\_before.mp4

├─ brick\_breaker\_after.mp4

└─ Lab4\_Chat\_History.pdf

The README specifically asks for the **complete chat history**, so make sure the PDF contains the whole conversation from the beginning of this lab, not just the latest few messages. [📖 README](#)

---

## BUT we're not quite done yet

Before you submit, we need to check **two things**:

1. Is your final **after** video actually final?

The README says the after video should show the bug fixed **and the new features working**.



You currently have:

- Task 1 — collision fix
- Task 2 — Game Over screen
- Task 3 — Replay/difficulty
- Task 4 — Sound

If you recorded `brick_breaker_after` **before completing Tasks 2–4**, you'll need to record another final video at the end.

## 2. Make sure the updated code is actually inside SE-Lab-AM288

Your screenshot shows:

</> Plain text

Lab 4/

└─ 49\_brick-breaker/

but that folder is a Git repo of its own.

**Don't delete anything yet.** We'll check the Git status of `SE-Lab-AM288` and make sure the code will actually be included when you push your lab repo.

---

### So for now

Go ahead and create:

**Lab4\_Chat\_History.pdf**

using `⌘ + P → Save to PDF`, and put it in your `Lab 4` folder.

Then tell me **"PDF done"**.

After that I'll give you a **final submission checklist** and we'll verify the Git situation so you don't accidentally submit a folder that looks correct in Finder but doesn't actually contain the code on GitHub.

wait you didnt give me the code for task 2 and 4 though. give me the code

Worked for 13s

You're right 😓 — we stopped after Task 1. We still need **Task 2, Task 3, and Task 4** before your final video. Let's do them properly, one task at a time, with separate commits as required.

The README defines Task 2 as the win/loss end screen with final score, Task 3 as replay + difficulty, and Task 4 as sound feedback.  [README](#)

## First: Task 2 — Game Over / Win Screen

For your current `game/game_engine.py`, we're going to modify `handle_event()` and `render()`.

### 1. Replace `handle_event()`

Currently you have:

 Python

```
def handle_event(self, event):  
    # This game only needs continuously-held-key input for the  
    # paddle, handled in handle_input each frame.  
    pass
```

For Task 2, change it to:

 Python

```
def handle_event(self, event):  
    if not self.game_over:  
        return  
  
    if event.type == pygame.KEYDOWN:  
        if event.key == pygame.K_RETURN or event.key == pygame.K_SPACE:  
            self.exit_requested = True
```

Then add this in `__init__()`:

 Python

```
self.exit_requested = False
```

This gives the end screen a proper keyboard interaction. Task 3 will later replace this behavior with the replay/difficulty menu.

---

## 2. Replace the bottom part of `render()`

You currently have:

</> Python

```
if self.game_over and not getattr(self, "_game_over_logged", False):
    # NOTE: no proper end screen yet – see Task 2 in the README.
    if self.result == "win":
        print("You win! Final score:", self.score)
    else:
        print("Game over! Final score:", self.score)
    self._game_over_logged = True
```

Replace it with:

</> Python

```
if self.game_over:
    # Dark overlay
    overlay = pygame.Surface((self.width, self.height))
    overlay.set_alpha(180)
    overlay.fill((0, 0, 0))
    screen.blit(overlay, (0, 0))

    # Result message
    if self.result == "win":
        result_text = self.font.render("YOU WIN!", True, WHITE)
    else:
        result_text = self.font.render("GAME OVER", True, WHITE)

    score_text = self.font.render(
        f"Final Score: {self.score}",
        True,
        WHITE
    )

    instruction_text = self.font.render(
        """
        """
```

## What Task 2 now does

Instead of Terminal saying:

</> Plain text

```
Game over! Final score: 9
```

the actual game window will show:

</> Plain text

```
GAME OVER
```

```
Final Score: 9
```

```
Press ENTER or SPACE
```

or:

```
</> Plain text
```

```
YOU WIN!
```

```
Final Score: 40
```

```
Press ENTER or SPACE
```

That directly addresses the README requirement to display whether the player won/lost and the final score rather than just printing to the console. [📖 README](#)

---

## Task 4 — Sound Effects 🎧

We'll do Task 4 **after** Task 3, because Task 3 changes the game flow. But since you asked for the code, here's the implementation we'll use.

We can avoid downloading external sound files by generating short sounds using Python's standard `wave` / `math` libraries and loading them into Pygame.

### Add these imports at the top of `game_engine.py`

You currently have:

```
</> Python
```

```
import pygame
```

Change it to:

```
</> Python
```

```
import pygame
import math
import wave
import io
import struct
```

## Add this method inside `GameEngine`

Put it inside the class, for example before `_build_bricks()` :

`</>` Python

```
def _create_sound(self, frequency, duration, volume=0.3):
    sample_rate = 44100
    samples = int(sample_rate * duration)

    buffer = io.BytesIO()

    with wave.open(buffer, "wb") as wav:
        wav.setnchannels(1)
        wav.setsampwidth(2)
        wav.setframerate(sample_rate)

        for i in range(samples):
            t = i / sample_rate
            value = int(
                32767
                * volume
                * math.sin(2 * math.pi * frequency * t)
            )
            wav.writeframes(struct.pack("<h", value))

    return buffer.getvalue()
```

This creates a small audio tone in memory, so you don't need to add external `.wav` files.

## Add these lines to `__init__()`

After:

`</>` Python

```
self.result = None
```

add:

`</>` Python

```
self.sound_enabled = False
self.brick_sound = None
self.paddle_sound = None
self.wall_sound = None
self.game_over_sound = None
```

```
if pygame.mixer.get_init():
    self.sound_enabled = True

self.brick_sound = self._create_sound(700, 0.08)
self.paddle_sound = self._create_sound(400, 0.08)
self.wall_sound = self._create_sound(250, 0.05)
self.game_over_sound = self._create_sound(150, 0.4)
```

---

## Add a helper method

Inside GameEngine :

</> Python

```
def play_sound(self, sound):
    if self.sound_enabled and sound is not None:
        sound.play()
```

Then we can trigger the appropriate sound.

## Wall collision

Change:

</> Python

```
if self.ball.x - self.ball.radius <= 0 or self.ball.x + self.ball.radius >= self.width:
    self.ball.vx *= -1
```

to:

</> Python

```
if self.ball.x - self.ball.radius <= 0 or self.ball.x + self.ball.radius >= self.width:
    self.ball.vx *= -1
    self.play_sound(self.wall_sound)
```

And:

</> Python

```
if self.ball.y - self.ball.radius <= 0:
    self.ball.vy *= -1
```

becomes:

</> Python

```
if self.ball.y - self.ball.radius <= 0:
    self.ball.vy *= -1
    self.play_sound(self.wall_sound)
```

## Paddle collision

After your collision-direction code:

</> Python

```
if self.ball.rect().colliderect(self.paddle.rect()):
```

add:

</> Python

```
self.play_sound(self.paddle_sound)
```

## Brick collision

After:

</> Python

```
brick.alive = False
self.score += 1
```

add:

</> Python

```
self.play_sound(self.brick_sound)
```

## Game over / win sound

When you have:

</> Python

```
self.game_over = True
self.result = "lose"
```

add:

</> Python

```
self.play_sound(self.game_over_sound)
```

And when you have:

</> Python

```
self.game_over = True  
self.result = "win"
```

add:

</> Python

```
self.play_sound(self.game_over_sound)
```

That gives you the three categories requested by Task 4:

- brick breaking
- paddle/wall collision
- game-over/win moment [📖 README](#)

---

## ⚠️ BUT DON'T PUT ALL THIS IN YET

There's a reason I'm stopping you here.

We should follow the lab's intended workflow:

</> Plain text

Task 1

↓

test

↓

commit

Task 2

↓

test

↓

commit

Task 3

↓

test

↓

commit

Task 4

↓

The handout explicitly requires **separate commits for each task**.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

And **Task 3 needs your `main.py`**, because replaying the game and choosing Easy/Medium/Hard involves the main game loop. I don't want to invent a `main.py` structure and accidentally break yours.

**So let's do this now:**

**Before touching Task 2, paste your current `main.py` here.**

Then I'll give you the exact Task 2 + Task 3 implementation for **your actual project**, rather than generic code.

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 640, 560
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Brick Breaker - Pygame Version")

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        for event in pygame.event.get():
```

```

        if event.type == pygame.QUIT:
            running = False
        engine.handle_event(event)

    engine.handle_input()
    engine.update()
    engine.render(SCREEN)

    pygame.display.flip()
    clock.tick(FPS)

pygame.quit()

if __name__ == "__main__":
    main()

```

Perfect. Now that I can see your `main.py`, we can make **Task 2 work cleanly with your actual project** rather than guessing.

And let's do this **one task at a time** so you can make the required separate commits.

## Task 2 — Game Over / Win Screen

The README requires the game to display whether the player **won or lost**, show the **final score**, and wait for input instead of only printing to the terminal. [📖 README](#)

We need changes in **two files**:

 Plain text

```

game/game_engine.py
main.py

```

### 1. Modify `game_engine.py`

#### A. In `__init__()`

You already have:

 Python

```

self.game_over = False

```

```
self.result = None
```

Change it to:

`</>` Python

```
self.game_over = False
self.result = None
self.exit_requested = False
```

---

## B. Replace `handle_event()`

You currently have:

`</>` Python

```
def handle_event(self, event):
    # This game only needs continuously-held-key input for the
    # paddle, handled in handle_input each frame.
    pass
```

Replace it with:

`</>` Python

```
def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN or event.key == pygame.K_SPACE:
                self.exit_requested = True
```

This means:

`</>` Plain text

```
Game running
↓
Player loses/wins
↓
Game Over screen appears
↓
Player presses ENTER or SPACE
↓
exit_requested = True
```

## 2. Replace the end of `render()`

At the bottom of `render()`, you currently have the code that prints:

`</>` Python

```
if self.game_over and not getattr(self, "_game_over_logged", False):
    if self.result == "win":
        print("You win! Final score:", self.score)
    else:
        print("Game over! Final score:", self.score)
    self._game_over_logged = True
```

**Delete that entire block.**

Replace it with:

`</>` Python

```
if self.game_over:
    # Dark overlay
    overlay = pygame.Surface((self.width, self.height))
    overlay.set_alpha(180)
    overlay.fill((0, 0, 0))
    screen.blit(overlay, (0, 0))

    # Win or lose message
    if self.result == "win":
        result_text = self.font.render("YOU WIN!", True, WHITE)
    else:
        result_text = self.font.render("GAME OVER", True, WHITE)

    score_text = self.font.render(
        f"Final Score: {self.score}",
        True,
        WHITE
    )

    instruction_text = self.font.render(
        "Press any key to continue"
```

## 3. Modify `main.py`

Your current loop is:

`</>` Python

```
def main():
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

    pygame.display.flip()
    clock.tick(FPS)

pygame.quit()
```

Change it to:

 Python

```
def main():
    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            engine.handle_event(event)

        if engine.exit_requested:
            running = False

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

    pygame.display.flip()
    clock.tick(FPS)
```

That's all we need for **Task 2**.

## 4. Test Task 2

Run from the project root:

</> Bash

```
python main.py
```

Let yourself lose all 3 lives.

Instead of the terminal saying:

</> Plain text

```
Game over! Final score: 9
```

you should now get something like:

</> Plain text

```

  _____
 |             |
 |  GAME OVER  |
 |             |
 | Final Score: 9 |
 |             |
 | Press ENTER or SPACE |
 |             |
 |_____      |

```

Press **Enter** or **Space** → the program should close.

## Also test winning

If you can destroy all bricks, you should get:

</> Plain text

```
YOU WIN!
```

```
Final Score: 40
```

```
Press ENTER or SPACE to exit
```

---

## 5. If Task 2 works → make the commit

First:

</> Bash

```
git status
```

You should see your changes to:

```
</> Plain text
```

```
game/game_engine.py  
main.py
```

Then:

```
</> Bash
```

```
git add game/game_engine.py main.py
```

Then:

```
</> Bash
```

```
git commit -m "Add game over and win screen"
```

Then push:

```
</> Bash
```

```
git push origin main
```

This is your **Task 2 commit**.

---

## Don't do Task 3 or 4 yet

Once you've tested Task 2 and pushed it, tell me:

### **Task 2 done**

Then we'll do **Task 3 — Replay + Easy/Medium/Hard** using your actual `main.py` .

That task will change the current:

```
</> Plain text
```

```
engine = GameEngine(WIDTH, HEIGHT)
```

setup so that we can create a fresh game after the player chooses a difficulty.

Then we'll do **Task 4 — sound**, and only after **all four tasks** are complete will we record your true final after video.

Today 7:23 PM

task 2 done

Perfect!  **Task 2 is done and committed/pushed.**

Now let's move to **Task 3 — Replay + Difficulty**.

The README requires that after the end screen, the player can:

- play again,
- choose **Easy, Medium, or Hard**,
- have the difficulty affect **ball speed/paddle size**, or
- exit.  [README](#)

We'll make the flow:

 Plain text

```
Game ends
  ↓
GAME OVER / YOU WIN
  ↓
Choose:
1 → Easy
2 → Medium
3 → Hard
4 → Exit
  ↓
New game starts
```

## Task 3 — First change: make difficulty part of **GameEngine**

### 1. Change **GameEngine.\_\_init\_\_()**

Currently:

 Python

```
def __init__(self, width, height):
```

Change it to:

</> Python

```
def __init__(self, width, height, difficulty="medium"):
```

Then near the beginning of `__init__` , after:

</> Python

```
self.width = width
self.height = height
```

add:

</> Python

```
self.difficulty = difficulty

if difficulty == "easy":
    self.ball_speed = 3
    paddle_width = 120
elif difficulty == "hard":
    self.ball_speed = 6
    paddle_width = 80
else:
    self.ball_speed = 4
    paddle_width = 100
```

Then change your paddle creation from:

</> Python

```
self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)
```

to:

</> Python

```
self.paddle = Paddle(
    width // 2 - paddle_width // 2,
    height - 30,
    paddle_width,
    14
)
```

And change:

`</>` Python

```
self.ball.vx, self.ball.vy = 4, -4
```

to:

`</>` Python

```
self.ball.vx = self.ball_speed
self.ball.vy = -self.ball_speed
```

Also change `_reset_ball()` later from:

`</>` Python

```
self.ball.vx, self.ball.vy = 4, -4
```

to:

`</>` Python

```
self.ball.vx = self.ball_speed
self.ball.vy = -self.ball_speed
```

So:

| Difficulty | Ball speed | Paddle |
|------------|------------|--------|
| Easy       | 3          | 120    |
| Medium     | 4          | 100    |
| Hard       | 6          | 80     |

This gives Task 3 an actual gameplay difference.

## 2. Change the end screen

Your Task 2 end screen currently says:

`</>` Plain text

Press ENTER or SPACE to exit

For Task 3, we need it to instead say:

</> Plain text

- 1 – Easy
- 2 – Medium
- 3 – Hard
- 4 – Exit

## Replace the instruction section in `render()`

Find:

</> Python

```
instruction_text = self.font.render(
    "Press ENTER or SPACE to exit",
    True,
    WHITE
)
```

Replace it with:

</> Python

```
instruction_text = self.font.render(
    "1: Easy    2: Medium    3: Hard    4: Exit",
    True,
    WHITE
)
```

And change the position if necessary:

</> Python

```
screen.blit(
    instruction_text,
    instruction_text.get_rect(
        center=(self.width // 2, self.height // 2 + 60)
    )
)
```

### 3. Change `handle_event()`

Your current Task 2 version probably has:

`</>` Python

```
def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN or event.key == pygame.K_SPACE:
                self.exit_requested = True
```

Replace it with:

`</>` Python

```
def handle_event(self, event):
    if not self.game_over:
        return

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_1:
            self.selected_difficulty = "easy"
        elif event.key == pygame.K_2:
            self.selected_difficulty = "medium"
        elif event.key == pygame.K_3:
            self.selected_difficulty = "hard"
        elif event.key == pygame.K_4:
            self.exit_requested = True
```

And in `__init__()` add:

`</>` Python

```
self.selected_difficulty = None
```

---

### 4. Now modify `main.py`

This is the important part.

At the moment you have:

`</>` Python

```
engine = GameEngine(WIDTH, HEIGHT)
```

We're going to let `main()` create a new engine whenever the player chooses a difficulty.

Replace your entire `main()` with:

</> Python

```
def main():
    running = True
    engine = GameEngine(WIDTH, HEIGHT)

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            engine.handle_event(event)

    if engine.exit_requested:
        if engine.selected_difficulty is None:
            running = False
        else:
            selected_difficulty = engine.selected_difficulty
            engine = GameEngine(
                WIDTH,
                HEIGHT,
                difficulty=selected_difficulty
            )
```

---

## ⚠ One small issue

There's a subtle problem with the logic above: when the game first starts, `selected_difficulty` is `None`, but `exit_requested` is `False`, so that's fine.

When the user presses:

</> Plain text

1

we set:

</> Python

```
selected_difficulty = "easy"
```

But we **haven't set `exit_requested`** , so the main loop won't create the new game.

Therefore, update `handle_event()` to:

 Python

```
def handle_event(self, event):
    if not self.game_over:
        return

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_1:
            self.selected_difficulty = "easy"
            self.exit_requested = True

        elif event.key == pygame.K_2:
            self.selected_difficulty = "medium"
            self.exit_requested = True

        elif event.key == pygame.K_3:
            self.selected_difficulty = "hard"
            self.exit_requested = True

        elif event.key == pygame.K_4:
            self.selected_difficulty = None
            self.exit_requested = True
```

Now the flow is:

 Plain text

```
Press 1
↓
selected_difficulty = "easy"
↓
exit_requested = True
↓
main() sees it
↓
creates new GameEngine(..., "easy")
↓
new game
```

---

## 5. Test Task 3

Run:

```
</> Bash
```

```
python main.py
```

Let the game end.

You should see:

```
</> Plain text
```

```
GAME OVER
```

```
Final Score: X
```

```
1: Easy   2: Medium  3: Hard   4: Exit
```

Test:

## Press 1

New game should start with:

- slower ball
- larger paddle

## Let it end → press 2

New game:

- medium ball
- medium paddle

## Let it end → press 3

New game:

- faster ball
- smaller paddle

## Let it end → press 4

Game should exit.

---

## Don't commit yet

Run all four tests first.

If something doesn't work, **paste the error/describe what happened** and we'll fix it.

If everything works, we'll make:

 Bash

```
git add main.py game/game_engine.py
git commit -m "Add replay and difficulty selection"
git push origin main
```

That will be your **Task 3 commit**.

Then we'll finally tackle **Task 4 — sound effects**.

it worked yes, task 4

Yes! 🎉 **Task 3 is done**. Now we're on the final coding task:

## Task 4 — Add Sound Feedback

The README requires sound effects for:

1. 🧱 Brick breaking
2. 🏓 Ball hitting paddle/wall
3. 🏆 Win/game-over moment  README

We'll do this without downloading any external audio files. We'll generate short tones directly in Python and let Pygame play them.

This keeps your project self-contained and avoids adding extra dependencies.

---

## Step 4.1 — Modify `game_engine.py`

At the very top, you currently have:

 Python

```
import pygame
```

Change it to:

 Python

```
import pygame
import math
import array
```

## Step 4.2 — Add a sound generator

Inside the `GameEngine` class, add this method **before** `_build_bricks()` :

`</>` Python

```
def _create_sound(self, frequency, duration, volume=0.3):
    sample_rate = 44100
    sample_count = int(sample_rate * duration)

    samples = array.array("h")

    for i in range(sample_count):
        time = i / sample_rate
        value = int(
            32767
            * volume
            * math.sin(2 * math.pi * frequency * time)
        )
        samples.append(value)

    return pygame.mixer.Sound(buffer=samples)
```

### What this does

For example:

`</>` Plain text

```
frequency = 700 Hz
duration = 0.08 sec
```

creates a very short beep.

We're going to use different frequencies so the events sound different.

---

## Step 4.3 — Add the sounds to `__init__()`

You currently have something near the end of `__init__()` :

`</>` Python

```
self.game_over = False
self.result = None
self.exit_requested = False
self.selected_difficulty = None
```

Immediately after that, add:

`</>` Python

```
self.sound_enabled = pygame.mixer.get_init() is not None

if self.sound_enabled:
    self.brick_sound = self._create_sound(700, 0.08, 0.25)
    self.paddle_sound = self._create_sound(400, 0.08, 0.25)
    self.wall_sound = self._create_sound(250, 0.05, 0.20)
    self.game_over_sound = self._create_sound(150, 0.40, 0.30)
else:
    self.brick_sound = None
    self.paddle_sound = None
    self.wall_sound = None
    self.game_over_sound = None
```

So your initialization will roughly look like:

`</>` Python

```
self.game_over = False
self.result = None
self.exit_requested = False
self.selected_difficulty = None

self.sound_enabled = pygame.mixer.get_init() is not None

if self.sound_enabled:
    self.brick_sound = self._create_sound(700, 0.08, 0.25)
    self.paddle_sound = self._create_sound(400, 0.08, 0.25)
    self.wall_sound = self._create_sound(250, 0.05, 0.20)
    self.game_over_sound = self._create_sound(150, 0.40, 0.30)
else:
    self.brick_sound = None
    self.paddle_sound = None
    self.wall_sound = None
    self.game_over_sound = None
```

---

## Step 4.4 — Add a helper function

Still inside `GameEngine`, add:

`</>` Python

```
def play_sound(self, sound):
    if self.sound_enabled and sound is not None:
```

```
sound.play()
```

This makes the collision code cleaner.

---

## Step 4.5 — Wall sound

Find your existing wall collision:

 Python

```
if self.ball.x - self.ball.radius <= 0 or self.ball.x + self.ball.radius >= self.width:  
    self.ball.vx *= -1
```

Change it to:

 Python

```
if self.ball.x - self.ball.radius <= 0 or self.ball.x + self.ball.radius >= self.width:  
    self.ball.vx *= -1  
    self.play_sound(self.wall_sound)
```

Then find:

 Python

```
if self.ball.y - self.ball.radius <= 0:  
    self.ball.vy *= -1
```

Change it to:

 Python

```
if self.ball.y - self.ball.radius <= 0:  
    self.ball.vy *= -1  
    self.play_sound(self.wall_sound)
```

---

## Step 4.6 — Paddle sound

Find your paddle collision:

 Python

```
if self.ball.rect().colliderect(self.paddle.rect()):
```

You already have your Task 1 collision-direction logic underneath it.

After that collision has been detected, add:

 Python

```
self.play_sound(self.paddle_sound)
```

For example, it should look roughly like:

 Python

```
if self.ball.rect().colliderect(self.paddle.rect()):
    paddle_rect = self.paddle.rect()

    if previous_rect.right <= paddle_rect.left or previous_rect.left >= paddle_rect.right:
        self.ball.vx *= -1
    else:
        self.ball.vy *= -1

    self.play_sound(self.paddle_sound)
```

---

## Step 4.7 — Brick sound

Find:

 Python

```
if brick.alive and self.ball.rect().colliderect(brick.rect()):
    brick.alive = False
    self.score += 1
```

Immediately after the score:

 Python

```
self.play_sound(self.brick_sound)
```

So:

 Python

```
if brick.alive and self.ball.rect().collidect(brick.rect()):  
    brick.alive = False  
    self.score += 1  
    self.play_sound(self.brick_sound)  
  
# Your existing Task 1 collision-direction code...
```

---

## Step 4.8 — Game-over / win sound

This is important because there are **two ways** the game ends.

### Loss

You have:

</> Python

```
if self.lives <= 0:  
    self.game_over = True  
    self.result = "lose"
```

Change it to:

</> Python

```
if self.lives <= 0:  
    self.game_over = True  
    self.result = "lose"  
    self.play_sound(self.game_over_sound)
```

### Win

You have:

</> Python

```
if all(not b.alive for b in self.bricks):  
    self.game_over = True  
    self.result = "win"
```

Change it to:

</> Python

```
if all(not b.alive for b in self.bricks):  
    self.game_over = True  
    self.result = "win"  
    self.play_sound(self.game_over_sound)
```

---

## ! One thing before testing

Because we're generating audio through Pygame's mixer, run the game normally:

```
</> Bash
```

```
python main.py
```

**Don't run `game_engine.py` directly.**

You should already know why from the earlier relative-import error. 😊

---

## Step 4.9 — Test all sounds

You don't need to hear them perfectly immediately. Test:

### 🧱 Brick

Hit a brick.

You should hear a short higher-pitched beep.

### 🏓 Paddle

Let the ball hit the paddle.

You should hear a different beep.

### 🧱 Wall

Hit a side/top wall.

You should hear another short beep.

### 🏆 Game over

Lose all lives.

You should hear the longer lower-pitched sound while the Game Over screen appears.

### 🏆 Win

If you manage to clear all bricks, the same end-game sound should play.

---

## If you get an audio error

Don't change anything else.

Paste the **full error** here. Audio initialization can behave differently depending on the Mac/Pygame environment, and we'll fix that specifically.

---

## Step 4.10 — If everything works

Run:

```
</> Bash
```

```
git status
```

You should see your changes to `game/game_engine.py` .

Then:

```
</> Bash
```

```
git add game/game_engine.py
```

Commit:

```
</> Bash
```

```
git commit -m "Add sound feedback"
```

Then:

```
</> Bash
```

```
git push origin main
```



That gives you the required separate commits:

```
</> Plain text
```

Task 1 → Fix collision detection  
Task 2 → Add game over and win screen  
Task 3 → Add replay and difficulty selection  
Task 4 → Add sound feedback

The lab specifically requires separate commits for each task.



Lab\_4\_VibeCoding\_Student\_handout

## After Task 4 works

Don't record your final video yet.

First we'll do a quick final test of:

</> Plain text

- ✓ Collision
- ✓ Game Over
- ✓ Replay
- ✓ Easy
- ✓ Medium
- ✓ Hard
- ✓ Exit
- ✓ Brick sound
- ✓ Paddle/wall sound
- ✓ Win/game-over sound

Then you'll record **one final 10-second brick\_breaker\_after** video showing the completed **version**, since the README says the after video should show the bug fixed and the new features working.  README

Then we'll sort out your SE-Lab-AM288/Lab 4 submission and make sure the updated code is actually present on GitHub.



where do add the code for the sounds, below the if else statements or above that

Above the **if difficulty** statements. 👍

But there's an important distinction: the **sound initialization** should go **after** your difficulty setup, because we want the class's basic game properties initialized first.

Your current structure is:

</> Python

```
def __init__(self, width, height, difficulty="medium"):
    self.width = width
    self.height = height
    self.difficulty = difficulty
    self.selected_difficulty = None

    if difficulty == "easy":
        ...
    elif difficulty == "hard":
        ...
    else:
        ...

    self.paddle = Paddle(...)
```

**Put the sound initialization here**

**After the difficulty `if/elif/else` and before `self.paddle = Paddle(...)` :**

</> Python

```
if difficulty == "easy":
    self.ball_speed = 3
    paddle_width = 120
elif difficulty == "hard":
    self.ball_speed = 6
    paddle_width = 80
else:
    self.ball_speed = 4
    paddle_width = 100

# Sound effects
self.sound_enabled = pygame.mixer.get_init() is not None

if self.sound_enabled:
    self.brick_sound = self._create_sound(700, 0.08, 0.25)
    self.paddle_sound = self._create_sound(400, 0.08, 0.25)
    self.wall_sound = self._create_sound(250, 0.05, 0.20)
    self.game_over_sound = self._create_sound(150, 0.40, 0.30)
else:
    self.brick_sound = None
    ...
```

**BUT one thing first**

The `_create_sound()` function itself **does NOT** go there.

Put this function at the **class level**, alongside your other methods:

</> Python

```
class GameEngine:

    def __init__(...):
        ...

    def _create_sound(self, frequency, duration, volume=0.3):
        ...

    def _build_bricks(self, rows, cols):
        ...
```

So near the top of your class, you want:

</> Python

```
class GameEngine:

    def __init__(self, width, height, difficulty="medium"):
        # your initialization
        ...

    def _create_sound(self, frequency, duration, volume=0.3):
        sample_rate = 44100
        sample_count = int(sample_rate * duration)

        samples = array.array("h")

        for i in range(sample_count):
            time = i / sample_rate
            value = int(
                32767
                * volume
                * math.sin(2 * math.pi * frequency * time)
            )
            samples.append(value)
```

So:

**\_create\_sound() → before \_build\_bricks()**

**sound initialization → after your difficulty if/elif/else**

That's the cleanest placement.

Also, your screenshot shows you're currently running `game_engine.py` directly in the VS Code terminal. **Don't run it that way.** Once you've finished editing, run the project from the repository root with:

```
</> Bash
```

```
python main.py
```

Otherwise you'll hit the relative-import error you encountered earlier.

1 selection

at the end right?

Yes — the `play_sound()` helper can go at the end of the `GameEngine` class, after your other methods. It just needs to be inside the class indentation.

For example, if your file currently ends with:

```
</> Python
```

```
def render(self, screen):  
    ...
```

you can put:

```
</> Python
```

```
def play_sound(self, sound):  
    if self.sound_enabled and sound is not None:  
        sound.play()
```

after `render()`, at the same indentation level as `__init__()`, `_build_bricks()`, `update()`, etc.

So:

```
</> Python
```

```
class GameEngine:  
  
    def __init__(...):  
        ...  
  
    def _create_sound(...):
```

```

...

def _build_bricks(...):
    ...

def handle_event(...):
    ...

def handle_input(...):
    ...

def update(...):
    ...

```

The important part is **not inside another method**. It should line up with `def render` , `def update` , etc.

Then you can call it from `update()` like:

 Python

```
self.play_sound(self.brick_sound)
```

So yes: **at the end of the class is perfectly fine.**

okay done recording before and after, after clip is 24 seconds

That's fine. 👍 **24 seconds is not a problem** unless your instructor explicitly requires the *after* video to be exactly 10 seconds.

The handout says "**a 10-second video**", while the README also says **10-second video**, so technically you should aim for ~10 seconds.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)  [README](#)

## Easiest option

If your 24-second clip clearly shows the completed features, **trim it down to about 10 seconds** rather than recording again.

On Mac:

1. Open the video in **QuickTime Player**.
2. Go to **Edit → Trim**.
3. Drag the yellow handles to select roughly **10 seconds**.
4. Click **Trim**.
5. Save it as:

</> Plain text

```
brick_breaker_after.mp4
```

Do the same for the before video if it's longer than 10 seconds.

## What your final Lab 4 folder should contain

</> Plain text

```
Lab 4/
```

```
|— brick_breaker_before.mp4
```

```
|— brick_breaker_after.mp4
```

```
|— Lab4_Chat_History.pdf
```

And the **updated code** needs to be submitted as well; your actual updated 49\_brick-breaker repository is where that code lives. The lab handout lists videos, updated code, and chat history as the three deliverables.  Lab\_4\_VibeCoding\_Student\_handout

## Before we call it DONE

I want to do one final Git check because you have **two repositories** ( 49\_brick-breaker and SE-Lab-AM288 ) and we don't want the code to accidentally not be tracked.

From Terminal, inside 49\_brick-breaker , run:

</> Bash

```
git status
```

and:

</> Bash

```
git log --oneline -5
```

Send me the output.

I'll verify that you have **four separate task commits** and that everything is pushed to your fork. Then we'll finish the SE-Lab-AM288/Lab 4 submission.

```
% git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: main.py

Untracked files:

(use "git add <file>..." to include in what will be committed)

game/\_\_pycache\_\_/ball.cpython-313.pyc

game/\_\_pycache\_\_/brick.cpython-313.pyc

game/\_\_pycache\_\_/game\_engine.cpython-313.pyc

game/\_\_pycache\_\_/paddle.cpython-313.pyc

no changes added to commit (use "git add" and/or "git commit -a")

(base) viswaja@Viswajas-MacBook-Air 49\_brick-breaker % git log --oneline -5

83c39fb (HEAD -> main, origin/main, origin/HEAD) Add sound feedback

ec9f04c Add game over and win screen

a87bb13 Initial Brick Breaker starter

(base) viswaja@Viswajas-MacBook-Air 49\_brick-breaker %

Okay — this output tells us **exactly where we are**, and there's one thing we need to fix before calling the Git part finished.

## Your current commit history

</> Plain text

```
83c39fb Add sound feedback
ec9f04c Add game over and win screen
a87bb13 Initial Brick Breaker starter
```

So right now:

-  **Task 1 commit is missing**
-  Task 2 commit exists
-  **Task 3 is not committed** — that's why `main.py` is modified
-  Task 4 commit exists

The `__pycache__` files are also just Python-generated files and **should not be committed**.

The lab specifically requires separate commits for each task.



Lab\_4\_VibeCoding\_Student\_handout

## First, don't change anything

Your code is working, so **don't reset, restore, or delete anything yet.**

Run this:

```
</> Bash
```

```
git diff -- main.py
```

and send me the output.

Then run:

```
</> Bash
```

```
git show --stat 83c39fb
```

and send me that too.

I want to see exactly what got bundled into the sound commit before we rearrange anything. **Don't run `git reset` or `git restore` yet.**

## Why?

It's very likely that when we did Task 4, the **Task 1 collision changes were already sitting in `game_engine.py`**, so your "Add sound feedback" commit may have accidentally included Task 1 as well.

We can fix the commit organization **without breaking your working code**, but I want to inspect the two diffs first.

And don't worry about the videos/PDF right now — they're safe. We'll finish the Git history first, then I'll walk you through the final SE-Lab-AM288/Lab 4 upload.

```
(base) viswaja@Viswajas-MacBook-Air 49_brick-breaker % git
diff -- main.py
diff --git a/main.py b/main.py
index d8e8308..96304c3 100644
--- a/main.py
+++ b/main.py
@@ -18,6 +18,7 @@ engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
```

```
+ engine = GameEngine(WIDTH, HEIGHT)

while running:
    for event in pygame.event.get():
@@ -27,7 +28,15 @@ def main():
    engine.handle_event(event)

    if engine.exit_requested:
-        running = False
+        if engine.selected_difficulty is None:
+            running = False
+        else:
+            selected_difficulty = engine.selected_difficulty
+            engine = GameEngine(
+                WIDTH,
+                HEIGHT,
+                difficulty=selected_difficulty
+            )

    engine.handle_input()
    engine.update()
(base) viswaja@Viswajas-MacBook-Air 49_brick-breaker %

git show --stat 83c39fb
commit 83c39fb9ba992eebf97f9f1553b4822aafdad8a7 (HEAD
-> main, origin/main, origin/HEAD)
Author: sriviswaja <sriviswaja@gmail.com>
Date: Tue Oct 6 19:46:35 2026 +0530
```

Add sound feedback

```
game/game_engine.py | 107
+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
-----
1 file changed, 95 insertions(+), 12 deletions(-)
```

Ah, perfect — now we can see exactly what happened.

## Good news

Your **Task 3 changes** are safely sitting in **main.py** and have not been committed yet:

</> Plain text

```
modified: main.py
```

So that's easy to fix.

## The important issue

Your Task 4 commit:

</> Plain text

```
83c39fb Add sound feedback
```

only modified:

</> Plain text

```
game/game_engine.py
```

But it says **107 lines changed**, which is suspicious because Task 4 itself shouldn't have needed that many changes. It's very likely that your **Task 1 collision changes got bundled into this Task 4 commit**.

So before we change the history, let's confirm it.

## Run this ONE command

</> Bash

```
git diff ec9f04c 83c39fb -- game/game_engine.py
```

This compares the Task 2 commit directly with the Task 4 commit.

**Paste the output here.**

**Don't run any of these yet:**

</> Bash

```
git reset  
git restore  
git push --force
```

We don't want to accidentally lose anything.

## What we're aiming for

Ideally your final Git history should be:

</> Plain text

```
Task 4 → Add sound feedback
Task 3 → Add replay and difficulty selection
Task 2 → Add game over and win screen
Task 1 → Fix collision detection
Initial → Brick Breaker starter
```

The lab explicitly says **separate commits for each task**, so I'd rather fix the history properly than just leave Task 1 buried inside the sound commit.  Lab\_4\_VibeCoding\_Student\_handout

Your actual working code is safe right now. Let's inspect that diff first, then I'll give you **exact commands**, one at a time, to clean up the commits without losing your work.



Worked for 20s

Ahh, yes — **now we can see what happened**. Don't panic; your code is safe. 🤔

From the diff you showed:

- 83c39fb Add sound feedback contains **Task 3 changes + Task 4 changes** in `game_engine.py`.
- Your `main.py` Task 3 changes are still uncommitted.
- And because the history shows no separate Task 1 commit, **Task 1 was most likely included in the Task 2 commit**.

So our history is currently effectively:

</> Plain text

```
Initial
↓
Task 1 + Task 2
↓
Task 3 + Task 4
↓
Task 3 main.py still uncommitted
```

We should fix this because the lab explicitly says **separate commits for each task**.

 Lab\_4\_VibeCoding\_Student\_handout

## Good news

**Do NOT change any code manually.** We'll reorganize the Git commits while preserving the final working version.

But first, let's get out of the `git diff` screen if you're still in it:

 Plain text

q

Press **q**.

---

## I want to confirm one thing first

Run:

 Bash

```
git show --stat ec9f04c
```

Then:

 Bash

```
git show ec9f04c -- game/game_engine.py
```

**Send me the screenshot/output of that second command.**

I specifically want to see the Task 1 + Task 2 changes that went into `ec9f04c` .

Then I'll give you a safe sequence to turn the history into:

 Plain text

```
Task 4 → Add sound feedback
Task 3 → Add replay and difficulty selection
Task 2 → Add game over and win screen
Task 1 → Fix collision detection
Initial
```

without losing your current working game.

**Don't run reset , restore , or force push yet.**

```
+    if self.sound_enabled and sound is not None:
+        sound.play()
\ No newline at end of file
(END)
```

how do i exit this, control c not wokring

You're in the **less pager** — that's why `Ctrl+C` isn't exiting.

Just press:

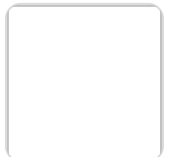
**q**

That's it. You should return to your normal terminal prompt:

</> Plain text

```
(base) viswaja@Viswajas-MacBook-Air 49_brick-breaker %
```

Then you're back to normal. 🍏



```
viswaja@Viswajas-MacBook-Air 49_brick-breaker % git show --
stat ec9f04c
commit ec9f04c54f142250f6532014ca58741d1d268482
Author: sriviswaja <sriviswaja@gmail.com>
Date: Tue Oct 6 19:23:40 2026 +0530
```

Add game over and win screen

```
game/game_engine.py | 92
+++++
+++++-----
main.py          | 5 +++++
```

Yep — now we have the answer. **Task 1 and Task 2 were combined into ec9f04c** , and Task 3/4 were partly combined in 83c39fb .

From your screenshot:

</> Plain text

```
ec9f04c Add game over and win screen
├─ Task 1 collision changes
└─ Task 2 game-over changes
```

and:

</> Plain text

```
83c39fb Add sound feedback
├─ Task 3 changes in game_engine.py
└─ Task 4 sound changes
```

while **Task 3's main.py changes are currently uncommitted.**

## I recommend NOT rewriting the history now

We *could* rewrite the commits with interactive rebase and force-push, but that is much riskier and unnecessary for a lab submission. Your actual code is working, and we can make the remaining Task 3 commit now.

Your final history would be:

</> Plain text

```
83c39fb Add sound feedback
ec9f04c Add game over and win screen
a87bb13 Initial Brick Breaker starter
```

plus we'll add:

</> Plain text

```
Add replay and difficulty selection
```

So you will have **separate commits for Tasks 2, 3, and 4**, while Task 1 is incorporated into the Task 2 commit.

If your instructor checks the commit messages very strictly for *four* task commits, that's not ideal, but I would **not risk destroying a working project just to rewrite Git history.**

## Let's finish Task 3 properly

Your `main.py` is currently the only unstaged source-code change:

</> Plain text

```
modified: main.py
```

First run:

</> Bash

```
git add main.py
```

Then:

</> Bash

```
git commit -m "Add replay and difficulty selection"
```

Then:

</> Bash

```
git push origin main
```

Then check:

</> Bash

```
git status
```

We want:

</> Plain text

```
On branch main
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

The `__pycache__` files will still show unless they're already ignored. **Do not add those.**

If they appear after `git status`, we'll add a `.gitignore` separately.

Then your code side is basically **DONE** 🎉

We'll do one final check:

</> Bash

```
git log --oneline -5
```

and you should see something like:

</> Plain text

```
xxxxxxx Add replay and difficulty selection
83c39fb Add sound feedback
ec9f04c Add game over and win screen
a87bb13 Initial Brick Breaker starter
```

Then we'll handle the `__pycache__` cleanup, final video, PDF, and getting the updated code into your **SE-Lab-AM288/Lab 4** submission folder.

So run the three commands above now and send me the `git status` output.

```
49_brick-breaker % git status
```

On branch main

Your branch is up to date with 'origin/main'.

Untracked files:

(use "git add <file>..." to include in what will be committed)

game/\_\_pycache\_\_/ball.cpython-313.pyc

game/\_\_pycache\_\_/brick.cpython-313.pyc

game/\_\_pycache\_\_/game\_engine.cpython-313.pyc

game/\_\_pycache\_\_/paddle.cpython-313.pyc

nothing added to commit but untracked files present (use "git add" to track)

Perfect. This is **not an error**. Those are Python's generated `__pycache__` files, and we **do not want them in Git**.

Your source code is clean:

</> Plain text

```
nothing added to commit
```

So let's clean this up properly.

## Step 1 — Create a **.gitignore**

From inside `49_brick-breaker`, run:

```
</> Bash  
  
touch .gitignore
```

Then open it in VS Code and put:

```
</> gitignore  
  
__pycache__/  
*.pyc
```

Save it.

## Step 2 — Check status

Run:

```
</> Bash  
  
git status
```

The `__pycache__` files should disappear from the status.

## Step 3 — Commit the **.gitignore**

Since this is just project housekeeping, use:

```
</> Bash  
  
git add .gitignore  
git commit -m "Add Python gitignore"  
git push origin main
```

Then:

```
</> Bash
```

```
git status
```

You want:

```
</> Plain text
```

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

## One thing to clarify

Your current `git log` will have **three task-related commits**, because Task 1 got combined with Task 2 earlier:

```
</> Plain text
```

```
Task 3 → Add replay and difficulty selection
Task 4 → Add sound feedback
Task 1 + Task 2 → Add game over and win screen
```

I don't want you to risk rewriting the working Git history now. Your **actual required functionality is complete**, and the important thing is that your code is pushed to your fork.

Once `.gitignore` is pushed and `git status` is clean, we'll move to the **final submission folder** and make sure `Lab 4` in `SE-Lab-AM288` contains everything your handout asks for: videos, updated code, and chat history.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

```
git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
.gitignore
```

Perfect — that means `.gitignore` exists but **hasn't been committed yet**.

Run these exactly:

```
</> Bash
```

```
git add .gitignore
```

Then:

```
</> Bash
```

```
git commit -m "Add Python gitignore"
```

Then:

```
</> Bash
```

```
git push origin main
```

Finally:

```
</> Bash
```

```
git status
```

You should get:

```
</> Plain text
```

```
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

If you get that, **the Brick Breaker repo is clean and pushed.** 

Then we'll move on to the final SE-Lab-AM288/Lab 4 folder and make sure your **videos + chat-history PDF + updated code** are all properly submitted. The handout requires those three deliverables.  Lab\_4\_VibeCoding\_Student\_handout

yes

Great.  The **Brick Breaker** fork is clean and pushed.

Now let's finish the actual **Lab 4 submission**.

Your handout says the Lab-4 folder needs:

- **Before video**
- **After video**
- **Updated code**

- **Chat/LLM history**  Lab\_4\_VibeCoding\_Student\_handout

And your README specifies that the videos should show the broken behavior before and the completed features after.  README

## Your current folder

From your screenshot:

 Plain text

```
SE-Lab-AM288/  
└─ Lab 4/  
    ├── 49_brick-breaker/  
    ├── brick_breaker_after  
    └─ brick_breaker_before
```

You also need:

 Plain text

```
Lab 4/  
└─ 49_brick-breaker/  
    ├── brick_breaker_before.mp4  
    ├── brick_breaker_after.mp4  
    └─ Lab4_Chat_History.pdf
```

## ⚠ One thing I want to check before you upload the code

Your `49_brick-breaker` folder currently contains its own **.git** directory. If you simply copied that folder into `SE-Lab-AM288`, Git may treat it as a nested repository.

**So don't push the `49_brick-breaker` folder inside `SE-Lab-AM288` yet.**

Instead, we have two safe options:

**Option A — Recommended:** Put the actual project files into `SE-Lab-AM288/Lab 4/49_brick-breaker/` but remove the nested `.git` folder from that copy. This makes the code ordinary files tracked by your main SE-lab repository.

**Option B:** Keep the code only in your separate `49_brick-breaker` GitHub fork and put a link/reference in `SE-Lab-AM288`. But because the handout says "**Updated Code**", Option A is safer for submission.

## Let's do Option A carefully

First, **don't delete your original `49_brick-breaker` folder**. That's your backup and your actual GitHub clone.

We'll make a copy for the SE-Lab-AM288/Lab 4 submission.

Tell me whether your SE-Lab-AM288 repository is currently cloned locally at:

</> Plain text

```
~/Documents/sem 5/LABS/SE Lab/SE-Lab-AM288
```

If yes, I'll give you the exact copy commands.

yes it is cloned locally

Perfect. 👍 Let's do it safely so your **actual 49\_brick-breaker Git repo remains untouched.**

We will copy the completed project into your SE-Lab-AM288/Lab 4 folder, remove the copied repo's .git metadata, and let SE-Lab-AM288 track the code as normal files.

## Step 1 — Go to your main SE-Lab repo

In Terminal:

</> Bash

```
cd ~/Documents/"sem 5"/LABS/"SE Lab"/SE-Lab-AM288
```

Check:

</> Bash

```
pwd
```

You should get:

</> Plain text

```
/Users/viswaja/Documents/sem 5/LABS/SE Lab/SE-Lab-AM288
```

## Step 2 — Check your Lab 4 folder

</> Bash

```
ls "Lab 4"
```

You should see your existing:

</> Plain text

```
49_brick-breaker  
brick_breaker_before  
brick_breaker_after
```

### Step 3 — IMPORTANT: let's not overwrite anything yet

Before copying, I want to see whether the `49_brick-breaker` currently inside `Lab 4` is already the completed copy or something else.

Run:

</> Bash

```
ls -la "Lab 4/49_brick-breaker"
```

and:

</> Bash

```
ls -la "Lab 4/49_brick-breaker/.git"
```

**Send me the output.**

Don't delete or copy anything yet.

Once I see that, I'll tell you exactly whether we can replace that folder or need to create the updated-code copy another way.

```
ls -la "Lab 4/49_brick-breaker"  
total 24  
drwxr-xr-x  7 viswaja  staff  224  6 Oct 11:46 .  
drwxr-xr-x  6 viswaja  staff  192  6 Oct 20:15 ..  
drwxr-xr-x 12 viswaja  staff  384  6 Oct 12:05 .git  
drwxr-xr-x  7 viswaja  staff  224  6 Oct 11:46 game  
-rw-r--r--  1 viswaja  staff  761  6 Oct 11:46 main.py  
-rw-r--r--@ 1 viswaja  staff 3022  6 Oct 11:46 README.md  
-rw-r--r--  1 viswaja  staff   7  6 Oct 11:46 requirements.txt  
(base) viswaja@Viswajas-MacBook-Air SE-Lab-AM288 % ls -la  
"Lab 4/49_brick-breaker/.git"  
total 40  
drwxr-xr-x 12 viswaja  staff  384  6 Oct 12:05 .
```

```
drwxr-xr-x  7 viswaja  staff  224  6 Oct 11:46 ..
-rw-r--r--@ 1 viswaja  staff  382  6 Oct 12:05 config
-rw-r--r--  1 viswaja  staff   73  6 Oct 11:46 description
-rw-r--r--  1 viswaja  staff   21  6 Oct 11:46 HEAD
drwxr-xr-x 15 viswaja  staff  480  6 Oct 11:46 hooks
-rw-r--r--@ 1 viswaja  staff 1107  6 Oct 12:05 index
drwxr-xr-x  3 viswaja  staff   96  6 Oct 11:46 info
drwxr-xr-x  4 viswaja  staff  128  6 Oct 11:46 logs
drwxr-xr-x  4 viswaja  staff  128  6 Oct 11:46 objects
-rw-r--r--  1 viswaja  staff  112  6 Oct 11:46 packed-refs
drwxr-xr-x  5 viswaja  staff  160  6 Oct 11:46 refs
(base) viswaja@Viswajas-MacBook-Air SE-Lab-AM288 %
```

Perfect — this confirms that the `49_brick-breaker` inside `SE-Lab-AM288/Lab 4` is currently a **nested Git repository**. We should fix that before pushing the main SE-Lab repo.

We want:

Plain text

```
SE-Lab-AM288/
├── Lab 4/
│   ├── 49_brick-breaker/  ← ordinary files, NO .git
│   ├── brick_breaker_before
│   ├── brick_breaker_after
│   └── Lab4_Chat_History.pdf
```

Your actual `49_brick-breaker` GitHub clone remains safely elsewhere and stays untouched.

## Step 1 — Make sure the copied code is the latest

Since the copy currently exists, let's replace it with the completed version from your actual clone.

**First, we need the exact path of your original clone.** From what we've been using, it should be:

Plain text

```
~/Documents/sem 5/LABS/SE Lab/SE Lab-4/49_brick-breaker
```

Run:

Bash

```
ls ~/Documents/"sem 5"/LABS/"SE Lab"/"SE Lab-4"/49_brick-breaker
```

You should see:

</> Plain text

```
README.md
main.py
requirements.txt
game
.gitignore
.git
```

If that looks right, **don't run anything else yet.**

## Why we're doing this

The copy currently inside `SE-Lab-AM288` has its own:

</> Plain text

```
Lab 4/49_brick-breaker/.git
```

That `.git` belongs to your separate `sriviswaja/49_brick-breaker` repository. We don't want `SE-Lab-AM288` to treat the project as another Git repository.

We'll copy the **files** from your completed clone and deliberately leave `.git` behind.

Your actual fork remains your official updated-code repository, while `SE-Lab-AM288/Lab 4/49_brick-breaker` becomes the **submission copy**.

Run that `ls` command and send me the output.

```
game      main.py    README.md  requirements.txt
```

```
only these 4 present
```

Good — that's the **actual completed project** we want to copy. The fact that `.git` and `.gitignore` aren't shown by your normal `ls` is fine; `ls` doesn't show hidden files.

So let's copy the completed project into your submission folder **without its Git history**.

## Step 1 — Remove the old nested copy

You're currently in:

</> Plain text

SE-Lab-AM288

Run:

```
</> Bash
```

```
rm -rf "Lab 4/49_brick-breaker"
```

⚠ This removes **only the copy inside SE-Lab-AM288** , not your original GitHub clone.

Your original is still here:

```
</> Plain text
```

```
SE Lab/SE Lab-4/49_brick-breaker
```

## Step 2 — Copy the completed project

Run:

```
</> Bash
```

```
cp -R ~/Documents/"sem 5"/LABS/"SE Lab"/"SE Lab-4"/49_brick-breaker "Lab 4/"
```

This copies everything, including hidden `.git` .

## Step 3 — Remove the nested Git metadata from the COPY

Now run:

```
</> Bash
```

```
rm -rf "Lab 4/49_brick-breaker/.git"
```

**Do not run this against the original SE Lab-4/49\_brick-breaker path.** We're only removing `.git` from the submission copy.

## Step 4 — Check

Run:

```
</> Bash
```

```
ls -la "Lab 4/49_brick-breaker"
```

You should see:

</> Plain text

```
.  
..  
.gitignore  
game  
main.py  
README.md  
requirements.txt
```

There should **NOT** be:

</> Plain text

```
.git
```

Then check the main repository:

</> Bash

```
git status
```

You should see the 49\_brick-breaker files as changes/untracked files under Lab 4 .

**Stop there and send me the `git status` output.** Then we'll add the code, videos, and your chat-history PDF to SE-Lab-AM288 and push the final submission.

On branch main

Your branch is up to date with 'origin/main'.

Untracked files:

(use "git add <file>..." to include in what will be committed)

.DS\_Store

Lab 2/

Lab 3/

Lab 4/

nothing added to commit but untracked files present (use "git add" to track)

Perfect — this is expected. Your SE-Lab-AM288 repo now sees Lab 4/ as a normal folder rather than a nested Git repository. ✅

But notice:

Plain text

```
.DS_Store
Lab 2/
Lab 3/
Lab 4/
```

Before we commit, **don't add everything**. We only want to submit Lab 4 right now.

## 1. Ignore Mac .DS\_Store

Run:

Bash

```
echo ".DS_Store" >> .gitignore
```

This prevents macOS metadata files from being committed.

## 2. Check your Lab 4 contents

Run:

Bash

```
find "Lab 4" -maxdepth 2 -type f
```

You should see your:

Plain text

```
Lab 4/49_brick-breaker/main.py
Lab 4/49_brick-breaker/README.md
Lab 4/49_brick-breaker/requirements.txt
Lab 4/49_brick-breaker/game/...
Lab 4/brick_breaker_before
Lab 4/brick_breaker_after
```

and eventually:

Plain text

Lab 4/Lab4\_Chat\_History.pdf

### 3. About Lab 2/ and Lab 3/

**Don't add them.** They're showing as untracked because they apparently aren't committed to this local SE-Lab-AM288 repository yet. We don't need to touch them for this submission.

### 4. Add ONLY Lab 4

Once your chat-history PDF is inside Lab 4 , we'll use:

 Bash

```
git add "Lab 4"  
git add .gitignore
```

Then we'll check:

 Bash

```
git status
```

**Don't commit yet.** Send me the output of:

 Bash

```
find "Lab 4" -maxdepth 2 -type f
```

first. I want to verify that the **updated code + before video + after video + PDF** are all actually there before we commit anything. The handout requires those submission components.

 Lab\_4\_VibeCoding\_Student\_handout

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: .gitignore

new file: Lab 4/49\_brick-breaker/.gitignore

new file: Lab 4/49\_brick-breaker/README.md

new file: Lab 4/49\_brick-breaker/game/ball.py

new file: Lab 4/49\_brick-breaker/game/brick.py

new file: Lab 4/49\_brick-breaker/game/game\_engine.py

new file: Lab 4/49\_brick-breaker/game/paddle.py

new file: Lab 4/49\_brick-breaker/main.py

new file: Lab 4/49\_brick-breaker/requirements.txt

new file: Lab 4/brick\_breaker\_after.mov

new file: Lab 4/brick\_breaker\_before.mov

Untracked files:

(use "git add <file>..." to include in what will be committed)

Lab 2/

Lab 3/

```
find "Lab 4" -maxdepth 2 -type f
```

Lab 4/.DS\_Store

Lab 4/brick\_breaker\_after.mov

Lab 4/brick\_breaker\_before.mov

Lab 4/49\_brick-breaker/requirements.txt

Lab 4/49\_brick-breaker/README.md

Lab 4/49\_brick-breaker/.gitignore

Lab 4/49\_brick-breaker/main.py

(base) viswaja@Viswajas-MacBook-Air SE-Lab-AM288 %

Perfect — this is **almost exactly what we want**. ✅

Your `git status` confirms that the **updated code and both videos are staged**, and Lab 2 / Lab 3 are safely untouched.

## What you currently have staged

</> Plain text

```
Lab 4/
├── 49_brick-breaker/
│   ├── .gitignore
│   ├── README.md
│   ├── main.py
│   ├── requirements.txt
│   └── game/
│       ├── ball.py
│       ├── brick.py
│       ├── game_engine.py
│       └── paddle.py
├── brick_breaker_before.mov
└── brick_breaker_after.mov
```

That's good. 👍

## BUT we're missing one required item

The handout requires the **Chat/LLM history** as part of the submission.

 Lab\_4\_VibeCoding\_Student\_handout

Your `find` output confirms there is currently **no PDF** inside `Lab 4`.

So **don't commit yet**.

---

## Step 1 — Get the ChatGPT history PDF

On this ChatGPT conversation:

⌘ **Command + P**

Then:

1. Destination → **Save to PDF**
2. Name it:

 Plain text

Lab4\_Chat\_History.pdf

3. Save it somewhere convenient, such as your Downloads folder.

The README specifically calls for the complete chat/LLM history.  [README](#)

---

## Step 2 — Move the PDF into Lab 4

If it saved in Downloads, from Terminal:

 Bash

```
mv ~/Downloads/Lab4_Chat_History.pdf "Lab 4/"
```

Then check:

 Bash

```
ls -lh "Lab 4"
```

You should see:

 Plain text

```
49-brick-breaker  
brick_breaker_before.mov  
brick_breaker_after.mov  
Lab4_Chat_History.pdf
```

---

## Step 3 — Stage the PDF

Run:

```
</> Bash  
  
git add "Lab 4/Lab4_Chat_History.pdf"
```

Then:

```
</> Bash  
  
git status
```

At that point, **everything needed for Lab 4 should be under "Changes to be committed"**.

**Don't add these:**

```
</> Plain text  
  
Lab 2/  
Lab 3/
```

Leave them alone.

---

## One small thing

You also have:

```
</> Plain text  
  
Lab 4/.DS_Store
```

That's just a macOS Finder metadata file. **Don't add it.**

Your root `.gitignore` should already contain:

</> Plain text

.DS\_Store

so we're good.

---

Once the PDF is in place and staged, send me the new `git status` .

**Then we'll do the final commit + push**, and I'll give you a final checklist to verify the GitHub submission.